

我们设计了一款卡牌游戏，、

一、棋盘格局为

- 左侧边：解锁的卡牌、进行的事件、回合进度、剩余遗物点、金币等等
- 右侧边：已获取的遗物（遗物槽数目一定）
- 中间核心区域：下半部分为己方棋盘，共三行，靠中间的两行各两格，最下面为一格，依次安排前排、法师和刺客、射手
- 下侧边：由卡牌分发的技能牌（技能槽数目一定）
可放置的手牌（数目一定）

示意图如下



二、游戏流程

- 每回合
 - 回合开始，获得金币，但不获得遗物点
 - |
 - 分发三种遗物（可以是特定回合再进行）
 - |
 - 在遗物点足够且遗物槽空置时，点击一个遗物可以购买到遗物槽
 - |
 - 分发四张（前中后排各一张，另一张随机）棋子牌
 - |
 - 在金币充足且棋子槽空缺时，点击一个棋子可将其购买到棋子槽
 - |
 - 放置棋子，点击选中的 n 个棋子将被放置到对应行
 - |
 - 此时棋盘棋子开始结算存活时间，如果到达一定数目将会在技能槽未满的条件下向技能槽分发棋子，同时分发顺序待定

遗物槽遗物开始计算效果

选择 n 个技能，他们将会被使用，并从技能槽消失

技能产生效果（展示双方使用技能，且双方同时进行）

结算效果

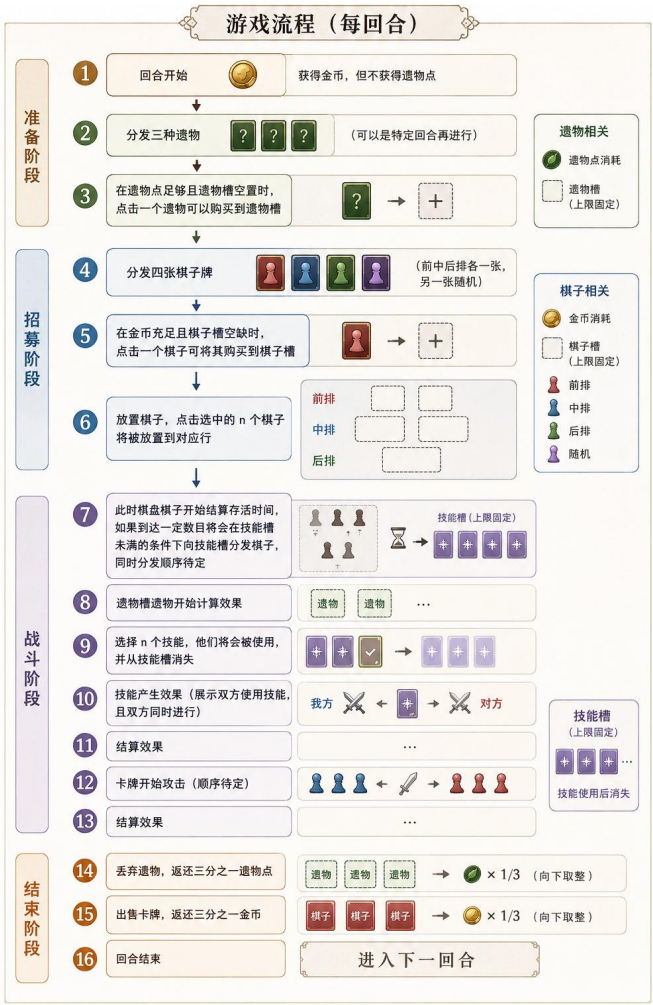
卡牌开始攻击（顺序待定）

结算效果

丢弃遗物，返还三分之一遗物点

出售卡牌，返还三分之一金币

回合结束



三、我期待的功能

回合结束时，如果棋子槽，技能槽，遗物槽出现三个一样的卡牌，下回合开始成为二星卡牌，能力获得提升

四、先实现简单功能

每回合

回合开始，获得 0 金币，但不获得遗物点

|

遗物忽略

|

分发四张（完全一样，各回合也一样的后排）棋子牌

|

在金币充足且棋子槽空缺时，点击一个棋子可将其购买到棋子槽

|

放置棋子，点击选中的 1 个棋子将被放置到对应行

|

技能忽略

|

卡牌开始攻击（顺序待定）

|

结算效果

|

出售和丢弃以及升星忽略

|

回合结束

|

金币无法再购买棋子且没有棋子判负，否则最大回合再结算

游戏流程（第一阶段·简化版）



首先是初级阶段的基础的 `cpp` 算法

第一页：选择游戏模式：点击右下方的三个按钮

、
分别是 `PVE`，`PVP`（不联网），`PVP`（联网）

鼠标点击按钮会为一个变量赋值，通过 `if` 判断进入不同游戏模式

不点对应按钮位置则继续点击按钮所在位置的循环

第二页：随之进入的游戏模式，这里我们先开发最简单版本（第二张流程图）的 `PVE`，也就是只有 `PVE` 有用，剩下的按钮暂时不起作用，也就是对应的 `if` 语句内为空白

我们默认 `PVE` 页面不需要选择大阵营，直接进入游戏流程

一、基础 `cpp` 架构：

1. 玩家状态储存，以 `if` 语句内的变量直接存储，分为大阵营（现在还没有），金币数，遗物点、当前总回合数（我们在 `PVE` 模式里不考虑它的增加，也就是不考虑总回合一定后结算胜负）

2. 遗物作为遗物类，右侧的五个遗物槽将作为五个遗物类的对象，由于遗物类的效果将是繁复的，因此遗物类没有派生类，且只存储一个字符串名字和所需遗物点，如果没有遗物则存储空字符串，由一个庞大的函数（输入为整个棋盘，也就是十个棋子类）负责结算遗物效果并输出

3. 棋子作为棋子类，棋盘是五个加五个（双方所有棋子），也就是二乘五维棋子类数组，棋子类成员变量存储信息为当前生命值、物理攻击力、法术攻击力、真实攻击力、物理防御（小数）、法术防御（同样是小数）、分发技能所需回合数、存活回合数计时器、前中后排标签、棋子自身名字，没有棋子的格子是空指针

4. 伪棋子类，只存储一个字符串名字、对应备战序号和所需金币，在备战区的双方棋子是它的对象，点击之后将通过另一个函数进行棋子类对象的赋值，没有的格子同样是空指针

5. 技能类，同样只存储一个字符串名字和对应备战序号，由函数扫描棋盘后为技能席的对象赋值，点击后由同样也由函数结算对棋盘的效果

二、理想流程

回合开始，获得金币，但不获得遗物点（直接改变 `if` 里的变量，也就是初版空置）

|

分发三种遗物（定义 `enum`，首先生成三个随机数在这个范围内，点击屏幕选择随机数，根据 `enum` 的对应关系，为第一个是空指针的遗物席位赋值，否则不赋值，以此作为分发遗物）

|

在遗物点足够且遗物槽空置时，点击一个遗物可以购买到遗物槽

|

分发四张（前中后排各一张，另一张随机）棋子牌（和遗物一样为伪棋子类的对象赋值）

|

在金币充足且棋子槽空缺时，点击一个棋子可将其购买到棋子槽

|

放置棋子，点击选中的 `n` 个棋子将被放置到对应行（设计一个函数，根据点击的伪棋子类，为场上的棋子类赋值）

|

此时棋盘棋子开始结算存活时间，如果到达一定数目将会在技能槽未满的条件下向技能槽分发棋子，同时分发顺序待定（根据函数扫描棋子类变量关系，对符合分发技能条件的，根据棋子名为技能类赋值）

|
遗物槽遗物开始计算效果（由函数扫描遗物类，根据遗物类名称执行对应效果）

|
选择 n 个技能，他们将会被使用，并从技能槽消失（此时创建新的技能类数组，并把点击的深拷贝进去，再有函数扫描他们执行对应效果）

|
技能产生效果（展示双方使用技能（也就是上一步的技能类组），且双方同时进行）

|
结算效果（包含死亡函数，也就是为所有生命值 ≤ 0 的赋值为空指针）

|
卡牌开始攻击（顺序待定）（扫描所有棋子，扫描到一个棋子后再根据前中后排标签确定攻击对象，对 A 攻击 B，B 受到 A 物理攻击力*B 物理防御+A 法术攻击力*B 法术防御+A 真实攻击）

|
结算效果（包含死亡函数，也就是为所有生命值 ≤ 0 的赋值为空指针）

|
丢弃遗物，返还三分之一遗物点（理论上和死亡函数同理，为了简单我们不让空的席位全在一侧了）

|
出售卡牌，返还三分之一金币

|
丢弃技能

|
回合结束（结算判断胜负函数）

二、第一阶段流程（和遗物、技能有关的函数和 `enum` 均先声明而不写）

第一阶段只开发一个后排怪物和一个玩家后排卡牌的对战，也就是对方的卡牌开始就赋值好了，我们不妨设为不分发技能，名字为“`testEnemy`”，初始生命值为 30，其余属性均为 0 的后排卡牌，而已方设置为名字为“`magicAnchor`”生命值为 20，物理和法术攻击力都是 1，剩下也为 0(伪棋子类购买金币也是 0)的后排卡牌，此时技能和卡牌的分发不再随机，因为各种 `enum` 大小要么是 1 要么是 0

回合开始，获得金币，但不获得遗物点（直接改变 `if` 里的变量，也就是初版空置）

|
遗物忽略

|
分发四张（等价于一张）棋子牌（和遗物一样为伪棋子类的对象赋值）

|
在金币充足且棋子槽空缺时，点击一个棋子可将其购买到棋子槽

|
放置棋子，点击选中的 1 个棋子将被放置到对应行（设计一个函数，根据点击的伪棋子类，为场上的棋子类赋值）

|
此时棋盘棋子开始结算存活时间，如果到达一定数目将会在技能槽未满的条件下向

技能槽分发棋子，同时分发顺序待定（这可以忽略了，大概声明好函数就行）

|

遗物槽遗物开始计算效果（忽略）

|

选择 n 个技能，他们将会被使用，并从技能槽消失（忽略）

|

技能产生效果（忽略）

|

结算效果（包含死亡函数，也就是为所有生命值 ≤ 0 的赋值为空指针）

|

卡牌开始攻击（顺序待定）（扫描所有棋子，扫描到一个棋子后再根据前中后排标签确定攻击对象，对 A 攻击 B ， B 受到 A 物理攻击力 $\times B$ 物理防御 $+A$ 法术攻击力 $\times B$ 法术防御 $+A$ 真实攻击）

|

结算效果（包含死亡函数，也就是为所有生命值 ≤ 0 的赋值为空指针）

|

丢弃遗物，返还三分之一遗物点（忽略）

|

出售卡牌，返还三分之一金币（忽略）

|

丢弃技能（忽略）

|

回合结束（这时的结算函数不需要根据回合数，只扫描棋盘的生命值计算胜负）

三、头文件结构

Game.h
Unit.h
PseudoUnit.h
Skill.h
Relic.h
Enums.h
Functions.h